Introduction to Computer Systems

Lecture 4: Processes and Threads

Comprehensive Lecture Note

Instructors: Wei Xu, Mingyu Gao

Teaching Assistants: Weitao Wan, Tingrui Zhang, Ruitao Liu, Shuo Chen

Spring 2026

Tsinghua University

Contents

1 Introduction **1**

2 Why Operating Systems Are Necessary **1**

3 Processes: The Fundamental Unit of Program Execution **2**

3.1 Program, Process, and Processor 2

3.2 Two Illusions Provided by a Process 3

3.3 The Reality Behind the Illusion 4

4 Threads, Execution Context, and Process State **5**

4.1 What a Thread Represents 5

4.2 Why Threads Matter 5

4.3 The Process Control Block 5

4.4 Context Switching 6

4.5 The Cost of a Context Switch 6

5 Address Spaces and Protection **7**

5.1 What an Address Space Provides 7

5.2 Threads Share an Address Space 8

5.3 A Simple Protection Mechanism: Base and Bound 8

6 User Mode, Kernel Mode, and Controlled Privilege **9**

6.1 Why Two Modes Exist 9

6.2 Three Ways to Enter the Kernel 10

6.2.1 System calls 10

6.2.2 Interrupts 10

6.2.3 Exceptions and traps 10

6.3 How the Kernel Handles Interrupts and Exceptions 11

6.4 Context Switches Revisited 12

7 Process Management in Unix-Style Systems **12**

7.1 Terminating a Process: `exit` 12

7.2 Creating a Process: `fork` 13

7.2.1 Independent address spaces after `fork` 14

7.2.2 Nested forks 14

7.3 Waiting for Children: `wait` and `waitpid` 14

7.3.1 Multiple children and nondeterminism 15

7.4 Orphans and the Special Role of `init` . . . 15
7.5 Replacing the Current Program: `exec` . . . 16
7.5.1 What survives an `exec`? . . . 17
7.6 Signals: Asynchronous Notifications . . . 17
7.7 The Shell as a User-Level Process Manager . . . 18

8 Scheduling: Choosing Who Runs Next **19**
8.1 Process and Thread Life Cycle . . . 19
8.2 Scheduling Goals and Metrics . . . 19
8.3 A Simplified Scheduling Model . . . 20
8.4 FCFS: First-Come, First-Served . . . 21
8.5 Round Robin . . . 22
8.5.1 Choosing the time quantum . . . 22
8.6 SJF and SRTF: Optimal Average Response Time . . . 23
8.6.1 Why SJF is optimal . . . 23
8.6.2 The fairness problem: starvation . . . 23
8.7 Comparing the Basic Policies . . . 24
8.8 Beyond the Toy Policies . . . 25

9 Summary **25**

1 Introduction

This lecture marks an important transition in the course. Earlier discussions emphasized the performance of a *single* running program: processor pipelines, memory technologies, caching, locality, and code transformations such as loop reordering and blocking. Those ideas answer questions such as: Why does one implementation of matrix multiplication run faster than another? Why does memory access pattern matter as much as arithmetic count? Why are cache-friendly loops often dramatically faster than pointer-heavy traversals?

The operating-system perspective asks a different question: what happens when *many* programs wish to use the same machine at the same time? A computer is not a laboratory setup in which one isolated program owns the processor and memory forever. A laptop runs an editor, a browser, a terminal, background daemons, and operating-system services simultaneously. A server may serve many users, each with multiple programs. Once a machine becomes a shared system, performance alone is no longer enough. The machine must also provide *coordination*, *protection*, *fairness*, and *controlled sharing*.

The core abstraction used to manage computation in an operating system is the *process*. A process gives a running program the illusion that it owns a processor and its own memory. This illusion is not accidental; it is carefully engineered through context switching, protection mechanisms, and operating-system policies. The related abstraction of a *thread* captures a schedulable stream of execution within a process. These abstractions are among the most fundamental ideas in computer systems because they connect hardware realities to software structure.

Deep Dive

The transition from memory hierarchy to processes is conceptually natural. Cache optimization studies how one program uses a shared memory hierarchy efficiently. Operating systems study how many programs share the *entire machine* efficiently and safely. In both cases, the system succeeds by exploiting structure: locality in one case, and controlled multiplexing plus protection in the other.

In this lecture note we will develop the following ideas in depth:

- why an operating system is necessary once multiple programs coexist;
- how a process differs from a program and from a processor;
- how threads, address spaces, and context switches create the illusion of exclusivity;
- how user mode and kernel mode enforce privilege separation;
- how key process-management system calls (`fork`, `exit`, `wait`, `exec`) work together;
- how signals and shells fit into the Unix process model;
- how basic scheduling policies trade off simplicity, fairness, and performance.

2 Why Operating Systems Are Necessary

In the simplified machine model used at the start of a systems course, one program runs on one processor and uses memory as though the entire computer were dedicated to it. This model is pedagogically useful because it isolates architectural ideas. It is also unrealistic.

A real computer must support multiple activities concurrently. Some are initiated directly by the user, such as compiling source code or opening a web browser. Others are initiated indirectly by the system, such as managing the display, maintaining network connections, flushing file-system buffers, or responding to input devices. If the machine is shared by several users, the problem becomes even more demanding: the hardware must be partitioned in a way that is correct, responsive, and safe.

An operating system (OS) exists to solve precisely this problem. At a high level, it has two complementary responsibilities.

First, it *controls hardware resources*. The processor, memory, storage devices, timers, and I/O devices are all finite resources. Someone must decide which program uses which resource, when, and for how long.

Second, it *coordinates resource use among competing activities*. If one process allocates memory, another process must not silently overwrite it. If two programs write to the same terminal or file, their access must be mediated. If a user presses `Ctrl-C`, the system must identify which process should receive that signal and what should happen next.

Definition

An **operating system** is the privileged software layer that manages hardware resources and provides controlled abstractions and services to user programs.

Operating systems therefore sit at the boundary between raw hardware capability and the programming interface seen by applications. They provide abstractions that are easier and safer to program against than direct hardware manipulation. The process is one such abstraction; virtual memory, files, sockets, and pipes are others.

3 Processes: The Fundamental Unit of Program Execution

3.1 Program, Process, and Processor

Confusion often arises because the words *program*, *process*, and *processor* sound similar while referring to entirely different entities.

A *program* is a passive object. It is a file: a compiled executable stored on disk. It contains instructions and perhaps associated metadata, but it is not yet doing anything.

A *processor* is hardware. It is the physical device that fetches, decodes, and executes instructions. In modern systems there may be multiple cores, but each core is still a physical execution engine.

A *process* is an active instance of a program in execution. The same program file can give rise to multiple processes. For example, two users may run the same editor binary; alternatively, one user may start the same browser program multiple times. Each running instance is a distinct process, with its own execution state and its own operating-system identity.

Definition

A **process** is an instance of an executing program. It is the operating system abstraction that gives a running program an execution context and an address space.

Table 1: Program, process, and thread compared

Term	Nature	Meaning
Program	Passive file	Executable code stored on disk; can be launched many times
Process	Active OS abstraction	Running instance of a program, with identity, state, and protected memory
Thread	Execution stream	A schedulable flow of control within a process; has registers, PC, and stack
Processor/Core	Hardware resource	Physical engine that executes instructions

3.2 Two Illusions Provided by a Process

A process provides two crucial illusions to software.

The first is the illusion of *exclusive use of the processor*. While a program runs, it appears as if it alone owns the CPU. Instructions execute in sequence, registers seem private, and the program counter advances through the process's own instruction stream.

The second is the illusion of *exclusive use of memory*. The program behaves as though it has its own private address space containing code, global data, heap allocations, and stack frames.

These are illusions in the strict technical sense: they are not literally true in the hardware, but the operating system makes them sufficiently true that software can reason as though they were true.

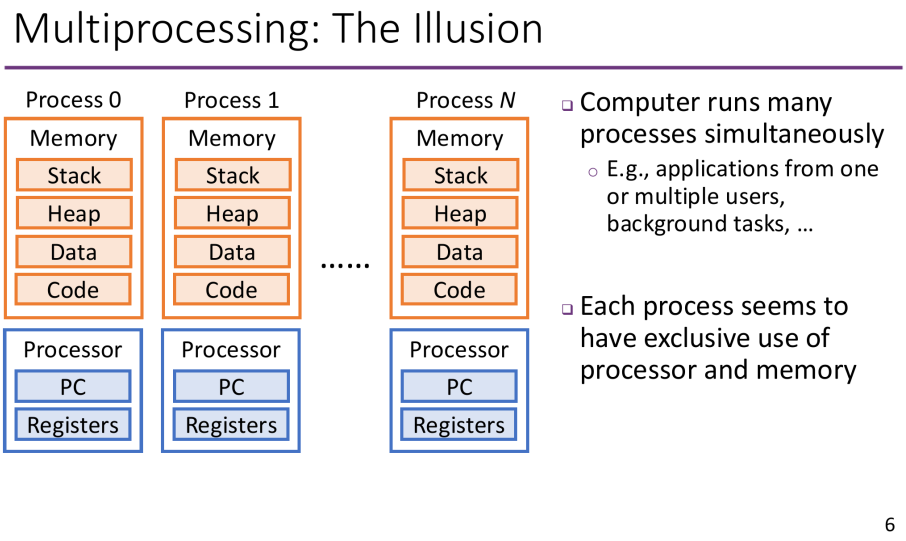


Figure 1: The abstraction seen by software: each process appears to own its own processor state and its own memory image, including code, data, heap, and stack. This is an intentionally simplified view that makes independent execution conceptually clean.

The illusion is especially powerful because it separates concerns. Application programmers usually do not need to reason about the low-level interleaving of all other programs on the machine. Instead, they write code within the abstraction of one process at a time.

3.3 The Reality Behind the Illusion

In reality, the machine's physical resources are shared.

Only one thread can execute on a single core at a given instant. When many processes exist, their executions are interleaved over time. One process runs briefly, then the operating system saves its state and switches to another. This *time sharing* creates the impression that many computations progress simultaneously.

Memory is also shared physically. Different processes' data reside in the same DRAM, but hardware and the operating system enforce isolation so that one process does not casually access another's memory. The state of a non-running process, including register values that are not currently resident in the core, is stored in memory until execution resumes.

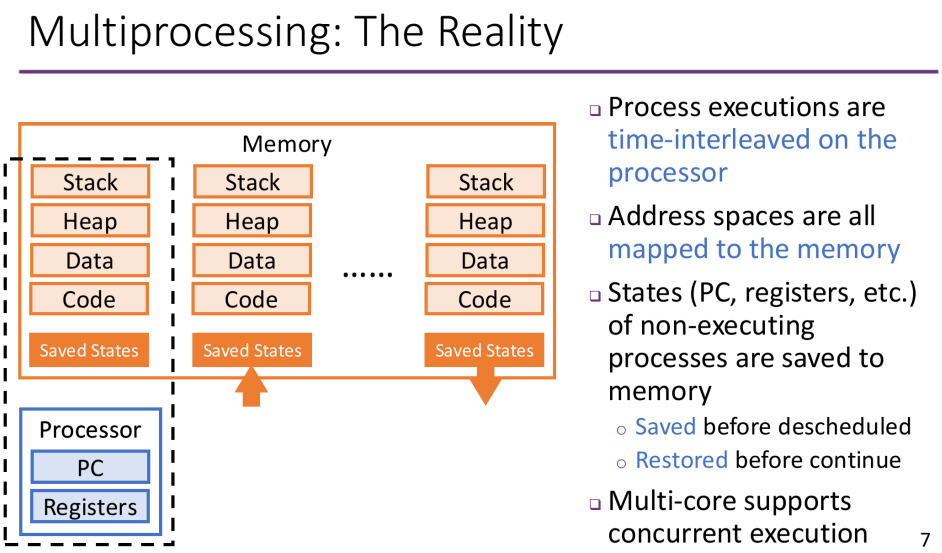


Figure 2: The underlying reality: processor execution is time-interleaved across processes, and the states of non-executing processes are saved in memory. Multi-core processors allow limited true concurrency, but even then the number of runnable execution contexts typically exceeds the number of cores.

A multi-core processor changes the degree, but not the principle. If a machine has four cores, then at most four threads can literally execute at once. If the system has hundreds of runnable threads, time sharing is still necessary.

Deep Dive

Time sharing is not free. Every context switch consumes cycles, perturbs pipeline state, and often degrades locality in the cache and TLB. This is one of the most important links between the current lecture and the earlier material on memory hierarchy: scheduling is about correctness and fairness, but it also has deep performance consequences.

4 Threads, Execution Context, and Process State

4.1 What a Thread Represents

A process may contain one thread or multiple threads. A *thread* is the unit of execution within a process: it is a particular control flow with its own current instruction, call stack, and register state.

Definition

A **thread** is a single execution context, consisting of the information required to resume execution of one control flow: typically the program counter, general-purpose registers, stack pointer, status flags, and associated scheduling state.

In a single-threaded process, the distinction between "process" and "currently running execution stream" is often blurred because there is only one thread. In a multithreaded process, however, the distinction becomes essential. The process provides the shared address space and shared resources; the threads provide multiple flows of control inside that common environment.

This design has an important consequence:

- Threads within a process can communicate efficiently because they share memory.
- Processes are better isolated from one another because their address spaces are separate.

That contrast reflects a deep trade-off between efficiency and protection.

4.2 Why Threads Matter

Threads are useful because many tasks naturally contain internal concurrency. A web server may want one thread to accept connections while others process requests. A scientific application may want multiple worker threads to process different subproblems in parallel. A graphical application may want one thread for the user interface and others for background work.

At the same time, threads are not merely a programming convenience; they are also the natural unit that the operating system scheduler can place on a CPU core. In modern systems, the object actually scheduled may be a kernel thread rather than an abstract process as a whole. At the conceptual level of this lecture, however, it is enough to understand that scheduling is fundamentally about choosing which execution context runs next.

4.3 The Process Control Block

The operating system must remember information about every process, even when that process is not currently running. This information is stored in an in-memory kernel data structure usually called the *process control block* (PCB).

Definition

A **process control block (PCB)** is the operating system data structure that records the state and metadata of a process, including its identifier, scheduling state, saved registers, memory-management information, and accounting information.

Typical PCB contents include:

- process identifier (PID);
- execution state such as running, ready, or blocked;
- saved register state;
- information about the address space;
- owner, privilege level, and resource permissions;
- scheduling metadata such as priority or accumulated runtime.

The scheduler manipulates collections of PCBs and associated queues. When a process is descheduled, relevant processor state is copied into its PCB or associated kernel structures. When the process runs again, that state is restored.

4.4 Context Switching

A context switch occurs when the operating system stops one execution context and later resumes another. This is central to time sharing.

Definition

A **context switch** is the act of saving the execution state of one thread or process and restoring the state of another, thereby transferring the processor from one execution context to another.

At minimum, a context switch must preserve enough state that the outgoing execution can later continue *as though nothing happened*. This generally includes the program counter, general-purpose registers, and stack pointer. Depending on the architecture and operating system, other state may also need attention: floating-point registers, vector registers, control registers, page-table references, and bookkeeping about privilege state.

Algorithm 1: Simplified Kernel-Mediated Context Switch

Input: Current thread T_{old}, next runnable thread T_{new}
Output: Processor executing T_{new}

1 Save processor registers, PC, and stack pointer into T_{old}'s saved state;
2 Mark T_{old} as ready, blocked, or terminated, depending on the event;
3 Select T_{new} from the scheduler's runnable set;
4 Load memory-management state associated with T_{new};
5 Restore processor registers, PC, and stack pointer from T_{new}'s saved state;
6 Mark T_{new} as running;
7 Return from kernel to user mode at T_{new}'s next instruction;
8 **return** T_{new};

4.5 The Cost of a Context Switch

People often understand context switching functionally but underestimate its performance cost. Saving and restoring a few registers is only part of the story.

A context switch may also incur:

- entry into and exit from the kernel;

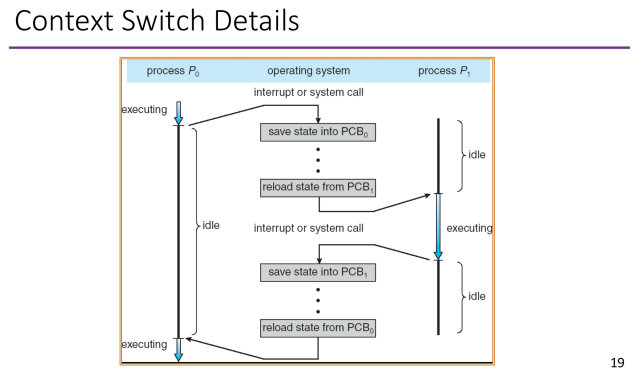


Figure 3: A detailed view of context switching: process P_0 executes, enters the operating system through an interrupt or system call, saves state into its PCB, and later the operating system restores the state of process P_1 from that PCB before resuming execution.

- scheduler computation and queue manipulation;
- TLB disruption when address spaces change;
- cache pollution when the incoming process has a different working set;
- reduced branch-prediction accuracy and lost pipeline state.

This is why operating systems do not switch frivolously. A scheduler must balance responsiveness against switching overhead. The same principle appears later in this lecture when discussing time quantum in Round Robin scheduling.

5 Address Spaces and Protection

5.1 What an Address Space Provides

Processes need more than execution state. They also need a protected memory abstraction.

Definition

An **address space** is the set of memory addresses a process can use, together with the mapping from those virtual addresses to actual memory and protection properties.

For a conventional user process, the address space includes several familiar regions:

- **text/code**: executable machine instructions;
- **static data**: global and static variables;
- **heap**: dynamically allocated memory such as objects obtained with `malloc` or `new`;
- **stack**: activation records for function calls, local variables, and return addresses.

Many systems also map a kernel region at high addresses, but that region is inaccessible while the processor runs in user mode.

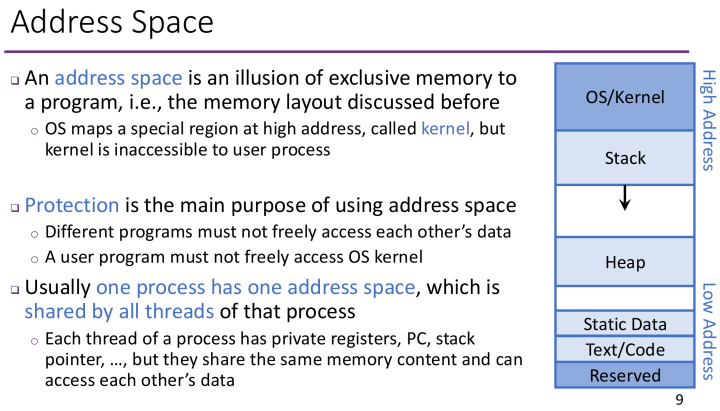


Figure 4: A typical conceptual address-space layout for a user process. Code and static data are placed near low addresses, the heap grows upward, the stack grows downward, and a protected kernel region occupies high addresses.

The purpose of an address space is not merely convenience. It is protection. A correct process should not be able to read or overwrite arbitrary data belonging to another process. A buggy or malicious program should not be able to seize kernel memory simply because it can manufacture a numeric address.

5.2 Threads Share an Address Space

All threads in the same process share the same address space. That means they can read and write the same global data, heap objects, and mapped regions. However, each thread has its own register state and its own stack.

This design explains both the power and the danger of multithreaded programming:

- it is powerful because communication is fast and natural through shared memory;
- it is dangerous because unsynchronized access to shared data can lead to races, corruption, and nondeterministic bugs.

The course will return to these issues when discussing concurrent programming in greater detail.

5.3 A Simple Protection Mechanism: Base and Bound

Before introducing full virtual memory, it is useful to consider a much simpler idea: each process is assigned a contiguous memory range, described by a base address and an upper bound. Hardware then checks every memory access against this permitted interval.

Base-and-bound is elegant as a teaching mechanism because it cleanly illustrates protection. Yet it has serious limitations.

First, it restricts each process to one contiguous memory block. Real systems often need more flexible allocation.

Second, dynamic growth is awkward. If a process's heap grows but the adjacent physical memory is occupied, simple expansion is impossible.

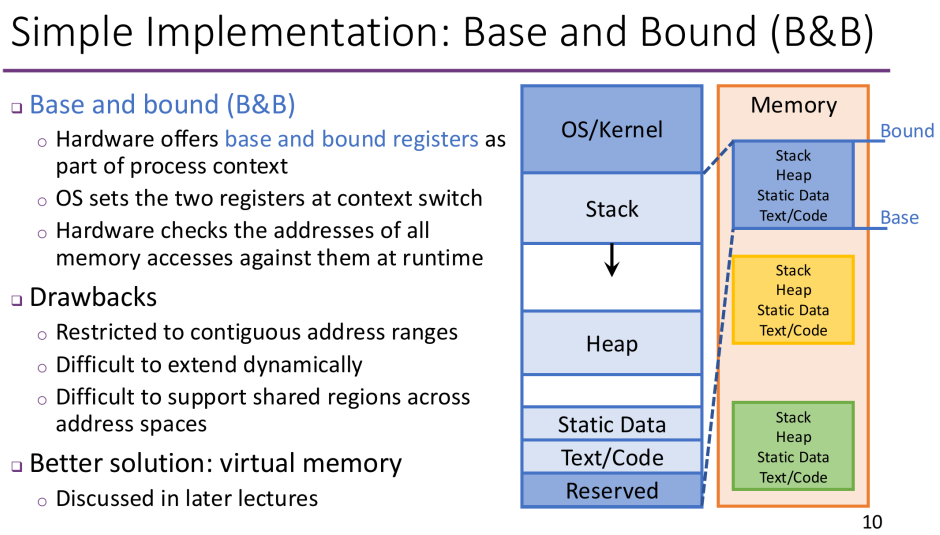


Figure 5: Base-and-bound protection: the operating system assigns each process a contiguous memory region, and hardware verifies that every user memory access falls within the valid range. The method is simple but inflexible.

Third, shared memory regions become cumbersome because the model assumes non-overlapping contiguous chunks.

These limitations motivate virtual memory, which will be studied later. Virtual memory preserves the abstraction of a private address space while removing the requirement that each process occupy a single contiguous range in physical memory.

Deep Dive

Base-and-bound is valuable conceptually because it separates two ideas that are often conflated:

1. *translation*: where a process's addresses map in physical memory; and
2. *protection*: whether an address is legal to access.

Virtual memory generalizes both ideas, but the simpler mechanism helps reveal the essence of isolation first.

6 User Mode, Kernel Mode, and Controlled Privilege

6.1 Why Two Modes Exist

If user programs had unrestricted access to all hardware instructions and all memory, the system would be unmanageable. Any ordinary program could disable interrupts, overwrite kernel data, reconfigure page tables, or interfere with other processes. A reliable multi-program system therefore requires privilege separation.

Definition

Kernel mode is the processor execution mode with high privilege, intended for the operating system. **User mode** is the restricted mode in which ordinary application code runs.

In kernel mode, sensitive operations are permitted. These include changing memory mappings, selecting the next process to run, or directly managing devices. In user mode, the program can execute ordinary instructions but cannot perform privileged actions directly.

Hardware support for this distinction usually includes:

- a mode bit or equivalent processor state indicating the current privilege level;
- instruction-set rules that allow certain operations only in kernel mode;
- mechanisms to transfer control safely from user mode to kernel mode and back.

6.2 Three Ways to Enter the Kernel

A user process must not jump into kernel mode arbitrarily. The transition must occur only through well-defined, safe mechanisms. This lecture emphasizes three such mechanisms.

6.2.1 System calls

A system call is an intentional request by the user program for an operating-system service. Examples include terminating the current process, creating a child process, writing to a file, allocating memory, or sending data through a network socket.

On modern architectures, a special instruction such as `syscall` (x86-64) or `ecall` (RISC-V) transfers control into the kernel. The kernel then inspects the request, validates arguments, performs the service if allowed, and returns to user mode.

6.2.2 Interrupts

An interrupt is triggered by an external event. A timer interrupt is the classic example used for preemptive scheduling: after a fixed interval, the timer forces the processor to enter the kernel so that the operating system can decide whether another process should run. Device events such as keyboard input or completion of a disk or network transfer are other common sources.

Interrupts are asynchronous with respect to the user program. The running process does not know precisely when they will occur.

6.2.3 Exceptions and traps

An exception arises from the current instruction stream itself. Divide-by-zero, illegal memory access, or execution of a forbidden instruction are standard examples. In Unix terminology, traps are often treated as a particular class of synchronous exceptions, especially those intentionally used for software requests.

Unlike interrupts, exceptions are synchronous with the faulting instruction: they occur because of something the program just attempted to do.

Table 2: Comparison of user-to-kernel transition mechanisms

Mechanism	Initiation	Timing	Typical examples
System call	Requested by program	Synchronous and intentional	`exit`, `fork`, `read`, `write`
Interrupt	External event	Asynchronous	Timer tick, keyboard input, device completion
Exception / trap	Caused by current instruction	Synchronous, often unexpected	Segmentation fault, divide-by-zero, invalid opcode

6.3 How the Kernel Handles Interrupts and Exceptions

Regardless of cause, the handling structure is similar. The processor stops ordinary user execution, saves enough state to allow recovery or diagnosis, and transfers control to kernel code. The kernel then selects an appropriate handler based on the cause. After processing the event, one of two things happens:

- the process resumes, either at the interrupted instruction or at the next one; or
- the process is aborted because the problem cannot be repaired safely.

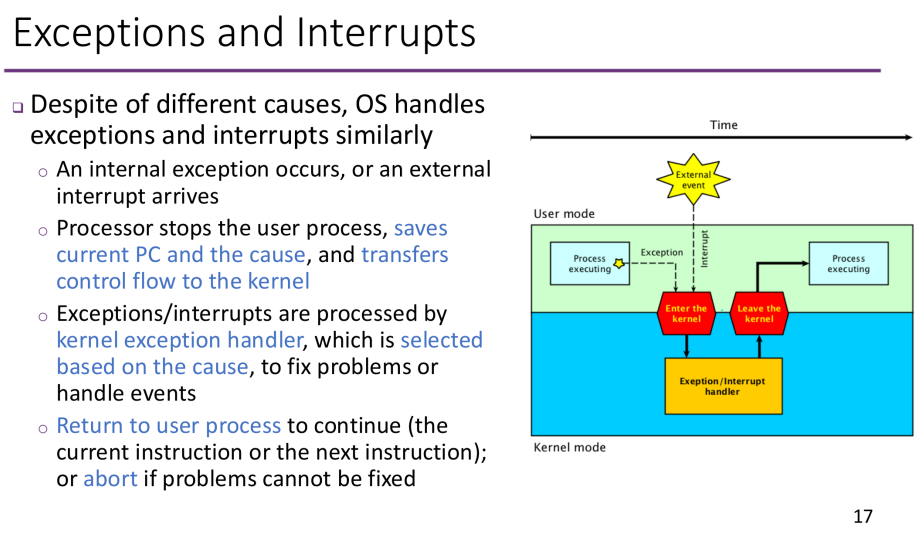


Figure 6: The general control-flow pattern for exceptions and interrupts: user execution is interrupted, control transfers to the kernel, an exception or interrupt handler runs, and the system either resumes the user process or aborts it.

A useful way to think about these transitions is that they resemble function calls and jumps, except that the control transfer crosses a privilege boundary. A system call is like a deliberate call into the operating system. An interrupt or exception is like an involuntary transfer that the program itself could not have scheduled in advance.

6.4 Context Switches Revisited

Because context switches are implemented by the operating system, they must themselves be reached through one of the above transition mechanisms.

Two common triggers are:

- a timer interrupt, which enables preemptive multitasking; and
- a system call such as `sleep` or an explicit yield-like operation, which voluntarily gives up the processor.

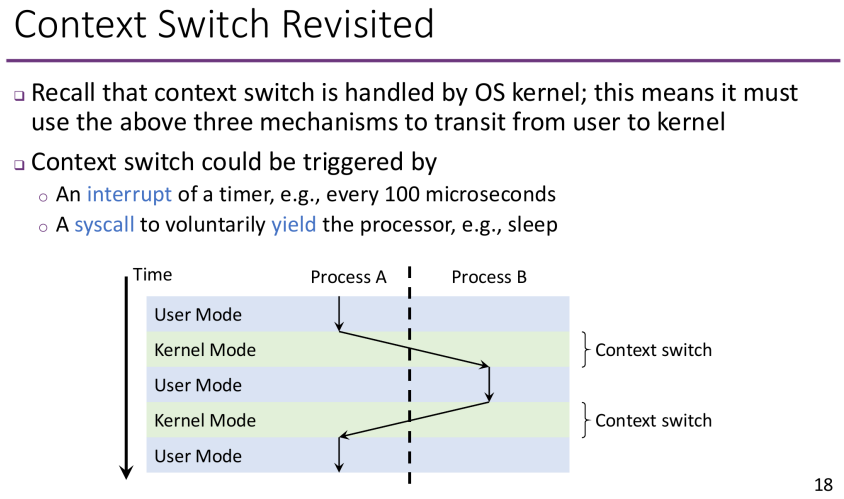


Figure 7: A context switch necessarily passes through the kernel: process A executes in user mode, enters kernel mode because of an interrupt or system call, and later the system resumes process B in user mode.

This is a subtle but important point. One does not switch directly from "user process A" to "user process B." The operating system stands in the middle, because only the kernel has the privilege to save and restore contexts, manage ready queues, and update accounting information.

7 Process Management in Unix-Style Systems

Unix-like operating systems expose process manipulation through a small but remarkably expressive set of system calls. Much of the elegance of Unix process management comes from how a few primitives combine to express a wide range of behavior.

7.1 Terminating a Process: `exit`

A process can terminate in several ways:

- normal return from `main`;
- explicit call to `exit(status)`;
- receipt of a signal whose default action is termination.

The key idea is that termination produces an *exit status*, typically with the convention that status 0 means success and nonzero values indicate some class of error. The operating system records this information so that the parent process can later inspect it.

One conceptual subtlety deserves emphasis: `exit` is called *once*, but it does not return to the program. It transfers control to the operating system, which tears down the process.

7.2 Creating a Process: `fork`

The `fork` system call creates a new child process by duplicating the calling process. The child is, at the abstraction level seen by the program, almost identical to the parent:

- it starts with the same code image;
- it receives a logically identical copy of the parent's address space;
- it inherits resource context such as open file descriptors and permissions;
- it has a different PID.

The most famous property of `fork` is this:

Definition

`fork()` is called once but returns twice: once in the parent and once in the child. In the child it returns 0; in the parent it returns the child's PID; on failure it returns a negative value.

This allows parent and child to distinguish their control paths immediately after the call.

Listing 1: A basic `fork` example: parent and child share initial values but diverge afterward

```c
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <unistd.h>

int main(void) {
    int x = 0;
    pid_t cpid = fork();

    if (cpid < 0) {
        perror("fork");
        return 1;
    } else if (cpid == 0) {
        // Child process
        printf("child pid=%d, x=%d\n", getpid(), --x);
    } else {
        // Parent process
        printf("parent pid=%d, child=%d, x=%d\n",
               getpid(), cpid, ++x);
    }
    return 0;
}
```

The order of the two `printf` calls after `fork` is not predictable. Once the child exists, the scheduler may run either process first.

7.2.1 Independent address spaces after `fork`

The variable `x` in the parent and the variable `x` in the child begin with the same value, but they are no longer the same object. Changes made by one process are not reflected in the other, because the processes have separate address spaces.

This is an important place where abstraction matters. A novice may think: "There was only one variable before the call, so there should still be only one after the call." The system call changes that premise. After `fork`, there are two processes, hence two logically separate address spaces.

Deep Dive

In modern operating systems, `fork` is usually implemented with *copy-on-write*. Rather than eagerly copying every memory page, the kernel lets parent and child temporarily share physical pages marked read-only. Only when one process writes to a page does the kernel create a private copy. This optimization preserves the semantics of independent address spaces while avoiding unnecessary copying cost.

7.2.2 Nested forks

Nested forks are conceptually simple but easy to misread because the code remains syntactically linear while execution branches into multiple processes. A good mental model is to draw a process tree and annotate which process executes which branch. After each successful `fork`, ask:

1. which process is the parent and which is the child?
2. what value does each process see as the return value?
3. which statements can each process execute next?

This disciplined tracing method is more reliable than trying to infer outcomes by inspection alone.

7.3 Waiting for Children: `wait` and `waitpid`

When a child process terminates, the kernel cannot immediately forget it entirely, because the parent may still want the child's exit status. The child therefore becomes a *zombie* until the parent reaps it.

Definition

A **zombie process** is a terminated process whose exit status and minimal bookkeeping information are still retained by the kernel because the parent has not yet collected them.

The parent uses `wait` or `waitpid` to collect that information. At that point the kernel can finally remove the zombie's remaining bookkeeping data.

Listing 2: Waiting for a child and checking its exit status

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <unistd.h>
```

```c

int main(void) {
    pid_t cpid = fork();

    if (cpid < 0) {
        perror("fork");
        return 1;
    } else if (cpid == 0) {
        printf("Hello from child\n");
        exit(7);
    } else {
        int status = 0;
        pid_t done = wait(&status);
        if (done > 0 && WIFEXITED(status)) {
            printf("Child %d exited with status %d\n",
                   done, WEXITSTATUS(status));
        }
    }
    return 0;
}
```

The meaning of `wait` is worth stating precisely. If no child has yet terminated, the parent blocks until one does. If a child has already become a zombie, `wait` can return immediately with that child's information.

7.3.1 Multiple children and nondeterminism

If a parent creates several children, they may terminate in any order, because scheduling and program behavior can differ. A loop that calls `wait` repeatedly should therefore not assume that children are reaped in creation order.

This reinforces a broader systems lesson: once concurrency exists, reasoning based solely on source-code order is often incorrect. The operating system determines interleavings, and code must be written to tolerate that fact.

7.4 Orphans and the Special Role of `init`

If a parent terminates before reaping a child, the child becomes an *orphan*. In traditional Unix-style systems, such children are adopted by a special process associated with PID 1, historically called `init`. That process eventually reaps them.

Definition

An **orphan process** is a process whose parent has already terminated. The system reparents it to a designated special process so that its eventual termination can still be handled cleanly.

This explains an important asymmetry:

- if the parent is long-running and never calls `wait`, zombie entries may accumulate;
- if the parent exits, the child is adopted and can later be reaped by the special system process.

This is why shells, servers, and other long-running programs must be careful to reap children explicitly.

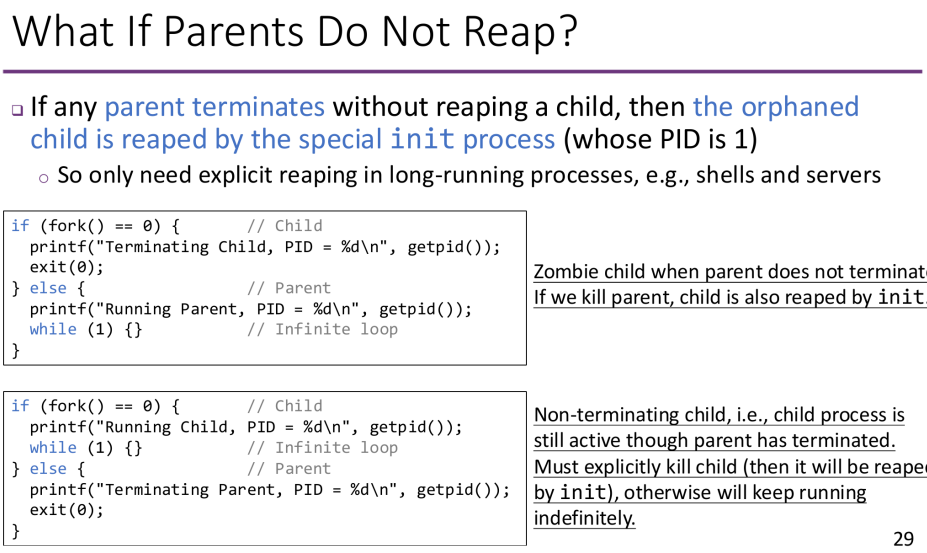


Figure 8: Two important corner cases. Top: if the child terminates but the parent keeps running without reaping, the child remains a zombie. Bottom: if the parent terminates first, the running child becomes an orphan and is later reaped by the special system process after it exits.

7.5 Replacing the Current Program: `exec`

So far, `fork` duplicates a process, meaning both parent and child are initially running the *same* program. In practice, the common pattern is to use the child to run a *different* executable. That is the role of the `exec` family.

Definition

The `exec` family of system calls replaces the current process image with a new program. The PID remains the same, but the code, data, heap, and stack are overwritten by the new executable.

If `exec` succeeds, control does not return to the old program text. The current process becomes the new program.

Listing 3: The classic fork-then-exec pattern used by shells

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <unistd.h>

int main(void) {
    pid_t cpid = fork();

    if (cpid < 0) {
        perror("fork");
        return 1;
    } else if (cpid == 0) {
```

```
14          char *argv[] = {"ls", "-l", NULL};
15          execv("/bin/ls", argv);
16
17          // execv returns only if it fails
18          perror("execv");
19          exit(1);
20      } else {
21          int status = 0;
22          waitpid(cpid, &status, 0);
23      }
24      return 0;
25  }
```

This pattern is so important that it deserves interpretation in words:

1. the parent creates a child with `fork`;
2. the child calls `exec` to become a different program;
3. the parent optionally waits for that child.

That is essentially how a shell launches commands.

Deep Dive

The elegance of `fork`-then-`exec` lies in separating concerns. `fork` establishes process structure and inherited context; `exec` establishes the new program image. This decomposition is one of the most influential design ideas in Unix systems.

7.5.1 What survives an `exec`?

Conceptually, the PID survives, because the process identity remains the same. The address space contents do not survive; they are replaced. In real Unix systems, some kernel-managed resources such as open file descriptors may also survive unless marked close-on-exec. This detail is central to features such as I/O redirection and pipelines, though those topics go beyond the present lecture.

7.6 Signals: Asynchronous Notifications

Signals allow the kernel, and sometimes other processes through the kernel, to notify a process that some event has occurred.

Examples include:

- `SIGINT`: typically generated by `Ctrl-C`;
- `SIGKILL`: forceful termination;
- `SIGSEGV`: invalid memory access;
- `SIGALRM`: timer expiration;
- `SIGCHLD`: child process terminated or changed state.

A received signal may have one of several effects:

- the default action occurs, such as termination;
- the signal is ignored;
- a user-installed signal handler runs.

Listing 4: Installing a simple signal handler with `sigaction`

```c
#include <signal.h>
#include <stdio.h>
#include <unistd.h>

static void handle_sigint(int sig) {
    (void)sig;
    write(STDOUT_FILENO, "caught SIGINT\n", 14);
}

int main(void) {
    struct sigaction sa;
    sa.sa_handler = handle_sigint;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = 0;
    sigaction(SIGINT, &sa, NULL);

    while (1) pause();
}
```

Deep Dive

Signal handlers are notoriously subtle. Because a signal may arrive asynchronously, only a restricted set of operations is safe inside a handler. In production Unix programming, one must pay close attention to so-called *async-signal-safe* functions. This is an example of how the clean abstraction of "small asynchronous notification" hides substantial implementation complexity.

7.7 The Shell as a User-Level Process Manager

A shell is not magic. It is itself a process. What makes it special is its role as a command interpreter and job-control system on behalf of the user.

When the user types a command, the shell typically:

1. parses the command line;
2. `fork`s a child;
3. lets the child `exec` the requested program;
4. either waits for the child (foreground job) or returns immediately (background job).

Foreground jobs are reaped directly by the shell. Background jobs require more care, because the shell often continues running for a long time. If it never reaps terminated background children, zombies can accumulate. Signals such as `SIGCHLD` and explicit job-control commands help manage this situation.

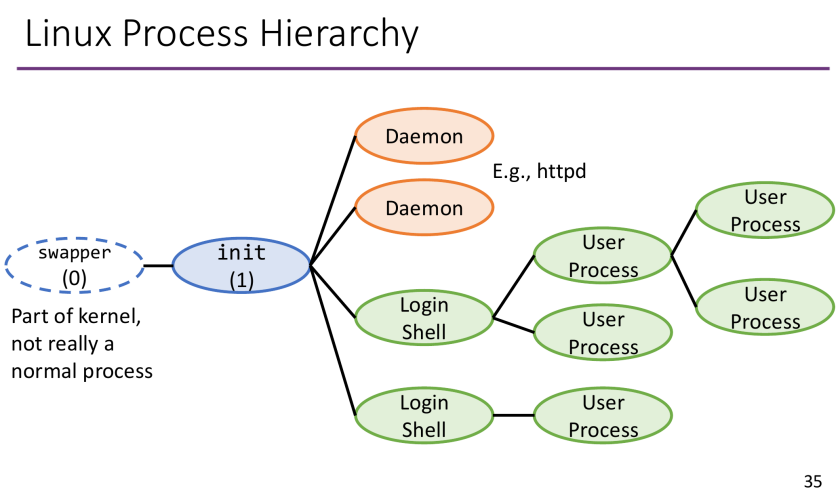


Figure 9: A conceptual Unix process hierarchy. The special process with PID 1 creates long-running system services and login shells; shells then create user processes. The figure highlights that shells are ordinary processes that manage other processes on the user's behalf.

8 Scheduling: Choosing Who Runs Next

8.1 Process and Thread Life Cycle

Once processes and threads exist, the system must manage their movement through execution states. A useful conceptual model includes the following states:

- **Ready**: eligible to run, waiting for a processor;
- **Running**: currently executing on a processor;
- **Suspended / Blocked**: not runnable until some event occurs;
- **Terminated**: finished execution.

The distinction between *ready* and *blocked* is particularly important. A ready process could run immediately if given a CPU. A blocked process cannot run yet because it is waiting for something external to complete: sleep timeout, I/O completion, lock release, child termination, and so on.

8.2 Scheduling Goals and Metrics

When several ready jobs compete for one processor, the scheduler must choose an order. There is no universally optimal policy, because different metrics capture different goals.

Definition

For the purposes of this lecture, **waiting time** is the time a job spends in the ready queue, and **response time** is waiting time plus execution time. Many textbooks call this latter quantity *turnaround time*; the lecture slides use the term response time for the total completion latency seen by the user.

We will use the following metrics:

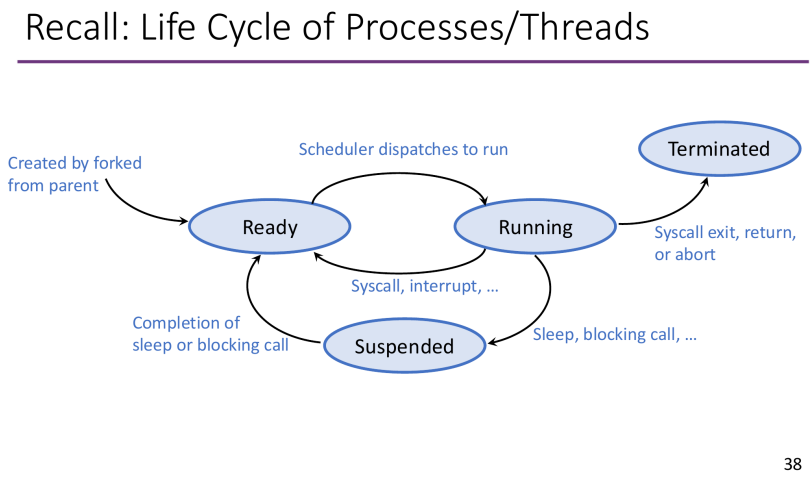


Figure 10: Life-cycle view of processes or threads. New execution contexts are created, become ready, may run, block, resume, and eventually terminate. Scheduling primarily concerns the transitions among ready, running, and suspended states.

- **Waiting time**: time spent ready but not running;
- **Response time**: total time from arrival until completion;
- **Throughput**: number of jobs completed per unit time;
- **Fairness**: how equally service is distributed among jobs.

These goals can conflict. A policy that minimizes average response time may be unfair to long jobs. A policy that is maximally fair may incur extra context switching and reduce throughput.

8.3 A Simplified Scheduling Model

To reason clearly about policies, it helps to adopt a simplified model:

- multiple independent jobs want service on one processor;
- jobs have no precedence constraints;
- jobs can be paused and resumed;
- jobs differ in execution time;
- all jobs have equal priority unless the policy says otherwise.

We also distinguish two broad classes of policy:

- **Non-preemptive**: once a job has been granted the processor for its assigned interval, the scheduler does not forcibly remove it before that interval ends.
- **Preemptive**: the scheduler may interrupt a running job and return it to the ready queue.

8.4 FCFS: First-Come, First-Served

FCFS, also known as FIFO, is the most intuitive policy. Jobs run in arrival order. Once a job starts, it runs until completion. This makes FCFS non-preemptive and simple to implement.

Suppose three jobs have execution times:

$$P_1 = 24, \qquad P_2 = 3, \qquad P_3 = 3.$$

If they arrive in the order P_1, P_2, P_3, then the schedule is:

$$P_1 \to P_2 \to P_3.$$

The waiting times are:

$$W_1 = 0, \qquad W_2 = 24, \qquad W_3 = 27.$$

Hence the average waiting time is

$$\frac{0 + 24 + 27}{3} = 17.$$

The response times are:

$$R_1 = 24, \qquad R_2 = 27, \qquad R_3 = 30,$$

so the average response time is

$$\frac{24 + 27 + 30}{3} = 27.$$

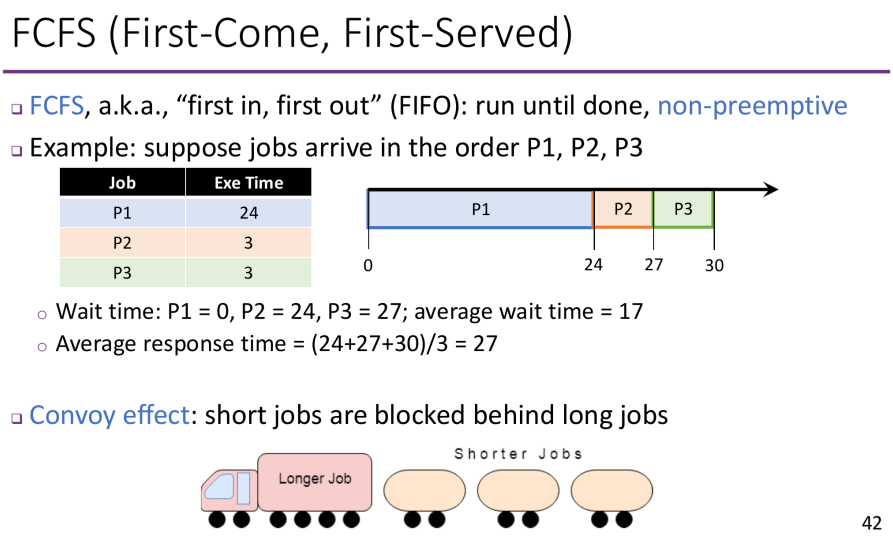


Figure 11: FCFS scheduling with one long job followed by two short jobs. The figure illustrates the convoy effect: short jobs wait behind a long one, inflating average waiting and response time.

The main weakness is the *convoy effect*: short jobs can get stuck behind a long one. If the arrival order changes to P_2, P_3, P_1, the average waiting and response times improve dramatically even though the jobs themselves are identical. Thus FCFS depends heavily on arrival order.

FCFS is attractive because it is simple and predictable, but it can perform poorly for mixed workloads.

8.5 Round Robin

Round Robin tries to improve fairness by giving each runnable job a limited slice of time, called the *time quantum*, before moving it to the back of the queue. This policy is preemptive.

Algorithm 2: Round Robin Scheduling

Input: Ready queue Q, time quantum q
Output: Repeated allocation of processor time

```
while Q is not empty do
    Remove the job at the head of Q;
    Run the job for at most q time units;
    if the job finishes during this slice then
        Record its completion time;
    else
        Save its updated state and append it to the tail of Q;
return completion order and completion times;
```

Consider the lecture example with time quantum $q = 20$ and four jobs:

$$P_1 = 53, \qquad P_2 = 8, \qquad P_3 = 68, \qquad P_4 = 24.$$

Each job runs for at most 20 time units before the scheduler rotates to the next ready job.

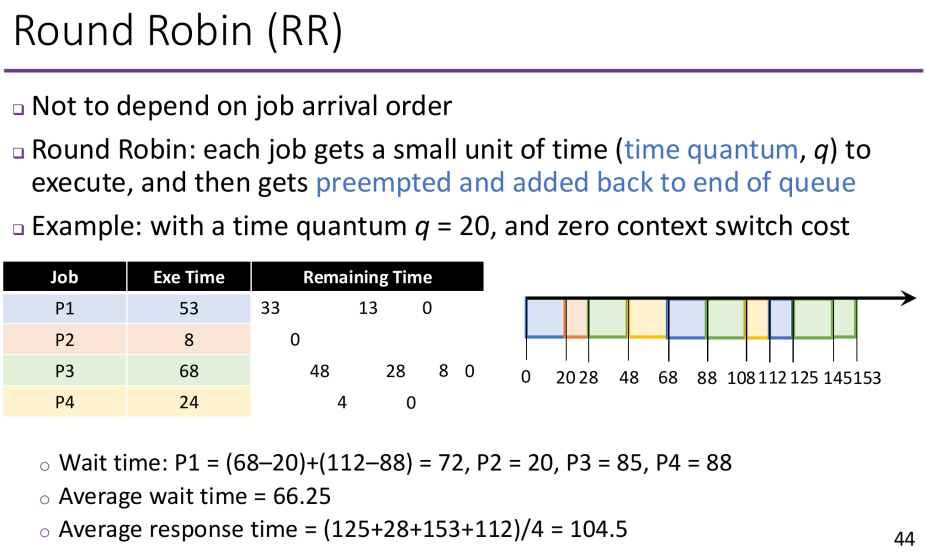


Figure 12: Round Robin execution with quantum $q = 20$. Each job receives a bounded amount of processor time before being preempted and returned to the queue if unfinished. The policy improves fairness but incurs switching overhead.

Round Robin is more fair than FCFS because short jobs are not forced to wait behind one long job until that long job finishes. Yet fairness comes at a cost: the system performs more context switches.

8.5.1 Choosing the time quantum

The time quantum is the key design parameter.

If the quantum is very large, Round Robin begins to resemble FCFS. Jobs effectively run to completion, and convoy effects reappear.

If the quantum is very small, the scheduler preempts frequently. The system becomes responsive, but a large fraction of processor time may be spent on context switching rather than useful computation.

Thus the quantum must be large enough that switching overhead is tolerable, yet small enough that interactive responsiveness remains acceptable. This is a classic systems trade-off.

Deep Dive

Round Robin also interacts with the memory hierarchy. A very small quantum can cause a thread to be preempted before it meaningfully exploits temporal locality, while the incoming thread may displace its cache working set. Thus the choice of quantum affects not only latency but also effective cache behavior.

8.6 SJF and SRTF: Optimal Average Response Time

Suppose, unrealistically but mathematically cleanly, that the scheduler knows every job's exact execution time in advance.

Then the optimal non-preemptive policy for minimizing average response time is *Shortest Job First* (SJF): run the shortest job first.

The optimal preemptive version is *Shortest Remaining Time First* (SRTF): always run the job whose *remaining* execution time is smallest; if a new shorter job arrives, preempt the current one.

8.6.1 Why SJF is optimal

A standard exchange argument proves the claim. Suppose two adjacent jobs i and j are scheduled in that order, with execution times t_i and t_j, and suppose $t_i > t_j$. Let T be the total time spent before either job starts.

If job i runs before job j, their contribution to total response time is:

$$(T + t_i) + (T + t_i + t_j) = 2T + 2t_i + t_j.$$

If we swap them so that j runs before i, the contribution becomes:

$$(T + t_j) + (T + t_j + t_i) = 2T + 2t_j + t_i.$$

Because $t_i > t_j$, we have

$$2T + 2t_j + t_i < 2T + 2t_i + t_j.$$

So placing the shorter job first strictly decreases the total response time. Repeating this adjacent-swap argument yields a schedule sorted by increasing job length. That is precisely SJF.

8.6.2 The fairness problem: starvation

SJF and especially SRTF can be unfair. A long job may wait indefinitely if short jobs keep arriving. This phenomenon is called *starvation*.

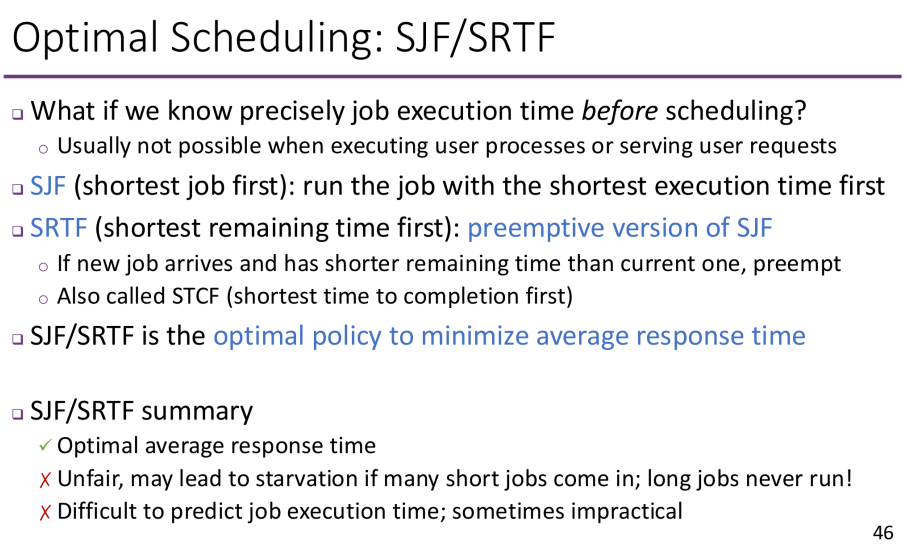


Figure 13: SJF and SRTF: if exact job lengths were known, shortest-job-first policies minimize average completion latency. Their weakness is not mathematical quality but practical difficulty and fairness concerns.

Definition

Starvation occurs when a runnable job is repeatedly denied service for so long that it effectively makes no progress, typically because the scheduling policy continually favors other jobs.

Thus a policy may be optimal under one metric and unacceptable under another. Operating systems are full of such trade-offs.

8.7 Comparing the Basic Policies

Table 3: High-level comparison of basic scheduling policies

Policy	Preemptive?	Strengths	Weaknesses
FCFS	No	Very simple, little scheduling overhead	Convoy effect, poor average latency when long jobs arrive early
Round Robin	Yes	Fairer sharing, good interactive feel	Context-switch overhead, sensitive to time quantum
SJF	No	Optimal average response time if lengths are known	Requires future knowledge, can be unfair
SRTF	Yes	Optimal average response time with preemption if lengths are known	Starvation risk, hard to predict remaining time

8.8 Beyond the Toy Policies

Real operating systems rarely use pure FCFS, pure Round Robin, or perfect SJF. Instead they combine ideas:

- priorities to distinguish classes of work;
- time slicing for responsiveness;
- dynamic adjustment based on observed behavior;
- fairness mechanisms to prevent starvation;
- load balancing across cores in multicore systems.

The simplified policies studied here are still invaluable because they isolate the central design tensions:

simplicity vs. performance vs. fairness.

9 Summary

Processes and threads are the fundamental abstractions through which operating systems present a usable machine to software.

A process is more than a running instruction stream. It combines an execution context with a protected address space. The execution context lets the process appear to own the processor; the address space lets it appear to own memory. Threads refine the execution side of this abstraction by allowing one process to contain multiple flows of control that share the same memory.

These abstractions only work because the operating system stands between user code and hardware. User mode and kernel mode enforce privilege separation. System calls, interrupts, and exceptions provide controlled transitions into the kernel. Context switching allows time sharing of the processor, while protection mechanisms ensure that memory sharing does not collapse into chaos.

On top of these foundations, Unix-like systems provide a small set of composable process-management primitives: `fork`, `exit`, `wait`, `exec`, and signals. The shell demonstrates how powerful these primitives are in practice by using them to launch and manage ordinary user programs.

Finally, once many runnable threads exist, the scheduler must choose who runs next. Even the simplest policies reveal the essential systems challenge: the "best" policy depends on what metric is most important. FCFS is simple, Round Robin is fairer, and SJF/SRTF is latency-optimal under strong assumptions. Real systems live in the space of trade-offs among all three ideas.